PROJE X: Asenkron E-Ticaret Fiyat Takip Motoru (Async Price Tracker)

Kazanım: I/O-bound işlerde thread açmadan tek iş parçacığında (event loop) yüzlerce ağ isteğini eşzamanlı yönetmek. Ürün fiyatlarını sırayla tek tek çekip her isteğin gecikmesini toplamak yerine, `asyncio` ile istekleri aynı anda uçurmak. Bunu yaparken karşı sunucuyu boğmamak için eşzamanlılığı sınırlamak (rate limiting), geçici hataları üstel geri çekilme (exponential backoff) ile tekrar denemek, farklı mağazaları ortak bir sözleşme (`Protocol`) üzerinden eklenti gibi takmak ve fiyat düşüşlerini kalıcı geçmiş üzerinden tespit etmek. (Python 3.11+)

Adım Adım Çözüm Algoritması

1. **Domain Modeli (Değişmez Veri):** Önce iş kavramları kodlanır, teknik detay sonra gelir. `Product(sku, name, source)` ve `PriceSnapshot(sku, price, currency, captured_at)` sınıfları `@dataclass(frozen=True, slots=True)` ile tanımlanır. `frozen` ile bir fiyat kaydı oluştuktan sonra değiştirilemez (tarihsel kayıt bozulmaz). `slots` ile örnek başına `__dict__` tahsis edilmez ve bellek kazanılır. Para tutarı `float` değil `Decimal` tutulur, çünkü `0.1 + 0.2 != 0.3` hatası fiyatta kabul edilemez.
2. **Kaynak Sözleşmesi (`typing.Protocol`):** Her mağaza farklı formatta veri döner. Bu yüzden ortak arayüz kalıtımla değil, yapısal tiplemeyle (duck typing + tip denetimi) tanımlanır:
   `class PriceSource(Protocol): async def fetch(self, product: Product) -> PriceSnapshot: ...`
   İki implementasyon yazılır: `MagazaA` JSON döner, `MagazaB` HTML benzeri metin döner ve fiyat regex ile ayıklanır. Kaynaklar parametreli bir decorator ile kayıt defterine (registry) eklenir: `@register_source("magaza_a")`. Yeni mağaza eklemek, mevcut koda dokunmadan yeni bir sınıf yazmaktan ibaret olur (Open/Closed prensibi).
3. **Sahte Ağ Katmanı (Deterministik Simülasyon):** Gerçek siteler bot engeli koyar ve sonuçlar tekrarlanamaz. Bu nedenle ağ `await asyncio.sleep(random.uniform(0.2, 1.0))` ile simüle edilir. İsteklerin %15'i rastgele `TransientError` (HTTP 429/503 karşılığı) fırlatır. `random.seed(42)` ile her çalıştırmada aynı senaryo oluşur, böylece ders boyunca herkes aynı çıktıyı görür.
4. **Yeniden Deneme Decorator'ı (Exponential Backoff + Jitter):** `@retry(attempts=3, base_delay=0.2, exceptions=(TransientError,))` şeklinde parametreli bir decorator yazılır (decorator factory, üç katmanlı fonksiyon yapısı). `functools.wraps` ile orijinal fonksiyonun adı ve docstring'i korunur. n. denemeden sonraki bekleme süresi:

$$\text{delay}_n = \text{base} \cdot 2^{\,n-1} + \mathcal{U}(0,\ \text{base})$$ Jitter terimi ($\mathcal{U}$), aynı anda hata alan yüzlerce isteğin aynı milisaniyede tekrar saldırmasını (thundering herd) engeller.

5. **Hız Sınırlama (Token Bucket):** `asyncio.Semaphore(N)` eşzamanlı istek sayısını sınırlar ama saniyedeki istek sayısını sınırlamaz. Bunun için kaynak başına bir token bucket kurulur. Kova kapasitesi b, dolum hızı r token/sn'dir. Her `acquire()` çağrısında kova güncellenir: $$\text{tokens} = \min\big(b,\ \text{tokens} + (t_{\text{şimdi}} - t_{\text{son}}) \cdot r\big)$$ Token yoksa `(1 - tokens) / r` saniye `await` edilir. Durum güncellemesi `asyncio.Lock` ile korunur. `threading.Lock` kullanılmaz, çünkü tek thread'de event loop'u bloklar.
6. **Zaman Aşımı ve Yapılandırılmış Eşzamanlılık:** Her istek `async with asyncio.timeout(3):` ile sınırlanır, böylece takılan bir sunucu tüm taramayı kilitlemez. Görevler `asyncio.TaskGroup` ile başlatılır. TaskGroup bir görev patlarsa diğerlerini iptal eder. Bu nedenle ürün bazlı hatalar görev içinde yakalanıp `FetchResult(ok=False, error=...)` olarak döndürülür ve tek bir ürünün hatası tüm taramayı düşürmez.
7. **Akış Halinde Sonuç (Async Generator):** `async def stream_prices(products)` fonksiyonu `asyncio.as_completed` ile hangi istek önce biterse onun sonucunu `yield` eder. Tüketici tarafı `async for snap in stream_prices(...)` ile sonuçları tamamı bitmeden işlemeye başlar. Tüm sonuçları listede bekletmek yerine akış modeli kullanılır ve bellek sabit kalır.
8. **Kaynak Yönetimi (Async Context Manager):** `PriceTracker` sınıfı `__aenter__` / `__aexit__` uygular. Girişte SQLite bağlantısı açılır ve tablo oluşturulur. Çıkışta, hata olsa bile commit/rollback yapılır ve bağlantı kapatılır: `async with PriceTracker("fiyat.db") as tracker:`. `sqlite3` bloklayan bir kütüphanedir. Event loop donmasın diye yazma işlemleri toplu halde (`executemany`) `await asyncio.to_thread(...)` ile ayrı thread'e devredilir.
9. **Fiyat Düşüşü Alarmı:** Her yeni fiyat, veritabanındaki aynı SKU'nun son kaydıyla karşılaştırılır: $$\Delta\% = \frac{p_{\text{eski}} - p_{\text{yeni}}}{p_{\text{eski}}} \times 100$$ $\Delta\% \ge$ eşik (örn. %10) ise `PriceAlert` nesnesi üretilir. Ek kural olarak fiyat, son 30 kaydın en düşüğünün altına inerse "tüm zamanların dibi" alarmı verilir.
10. **Ölçüm ve Kıyas (`@timed` Decorator):** Aynı 50 ürün önce senkron döngüyle, sonra async motorla çekilir. Sürelerin teorik değerleri: $$T_{\text{sync}} \approx \sum_{i=1}^{n} \text{latency}_i \qquad T_{\text{async}} \approx \left\lceil \frac{n}{N_{\text{eşzamanlı}}} \right\rceil \cdot \max(\text{latency})$$ Ölçülen değerler teoriyle karşılaştırılır (beklenen sonuç: ~30 sn → ~2-3 sn). Retry ve rate limit eklenince aradaki farkın nereden geldiği tartışılır.

Dosyalar:

- **`fiyat_takip.py`**: referans çözüm. Kod `ADIM 1` … `ADIM 10` başlıklarıyla bölümlendi, her bölümün başında anlatım notu var.
- **`fiyat_takip_iskelet.py`**: öğrenciler için. Sahte ağ, SQL ve ana akış hazır, 24 `TODO` doldurulacak.
- **`test_fiyat_takip.py`**: 25 kabul testi. Referans çözümde hepsi geçiyor. İskelete karşı da çalışıyor: `FIYAT_MODUL=fiyat_takip_iskelet`.
- **`README.md`**: komutlar, dakika bazlı ders akışı, 5 tartışma sorusu.

Doğrulanan canlı demo çıktıları (seed=42, her çalıştırmada aynı sonuç):

- 3 tur tarama. Sonuçlar bitiş sırasıyla akıyor, bu yüzden SKU sırası karışık geliyor (`as_completed` görsel olarak görülüyor).
- 404 alan ürün retry edilmiyor. 429/503 hataları ve timeout retry ediliyor (29 retry).
-
    - turda `FİYAT DÜŞTÜ`, 3. turda `YENİ DİP` alarmları geliyor.
- `--bench` kıyası: eşzamanlılık 1 ile **14.3 sn**, 10 ile **1.95 sn** (7.3x hızlı).
- Bloklama tuzağı: `asyncio.sleep` ile 0.30 sn, `time.sleep` ile 3.00 sn.
- `--rate 3` verilince token bucket devreye giriyor (`rate_limit_bekleme=51`).

Dersin en güçlü anları:

1. Retry'da `except Exception` kullanılıyor ama `CancelledError` buna yakalanmıyor, çünkü `BaseException`'dan türüyor.
2. `aclosing` kaldırılırsa `stream()` içindeki `finally` bloğu hemen çalışmıyor.
3. `--bench` sonucundaki 14 sn → 2 sn farkı.